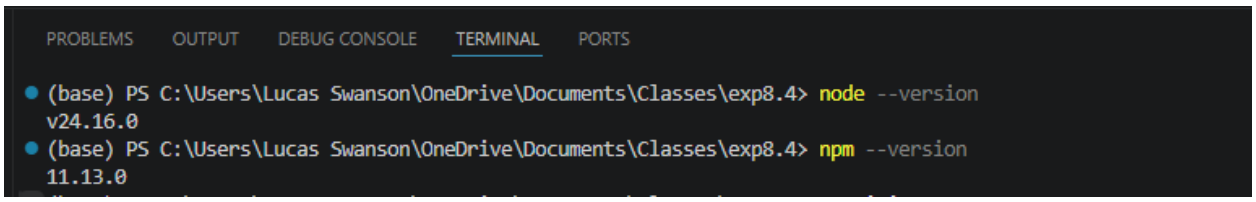
Lucas Swanson
5/26/2025
Software Engineering Methods

Homework 8 (Playwright)

1. Setup:

The first part of setting up the tool is ensuring all dependencies are installed, this includes NPM, or Node. Node is a javascript runtime and npm is a package manager for Node.js.



Next you need to initialize npm within the current workspace so it can manage the dependencies for the packages used:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(base) PS C:\Users\Lucas Swanson\OneDrive\Documents\Classes\exp8.4> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (exp8.4)
npm warn init canceled
npm notice
npm notice New minor version of npm available! 11.13.0 -> 11.15.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.15.0
npm notice To update run: npm install -g npm@11.15.0
npm notice
(base) PS C:\Users\Lucas Swanson\OneDrive\Documents\Classes\exp8.4> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields and exactly what they do.

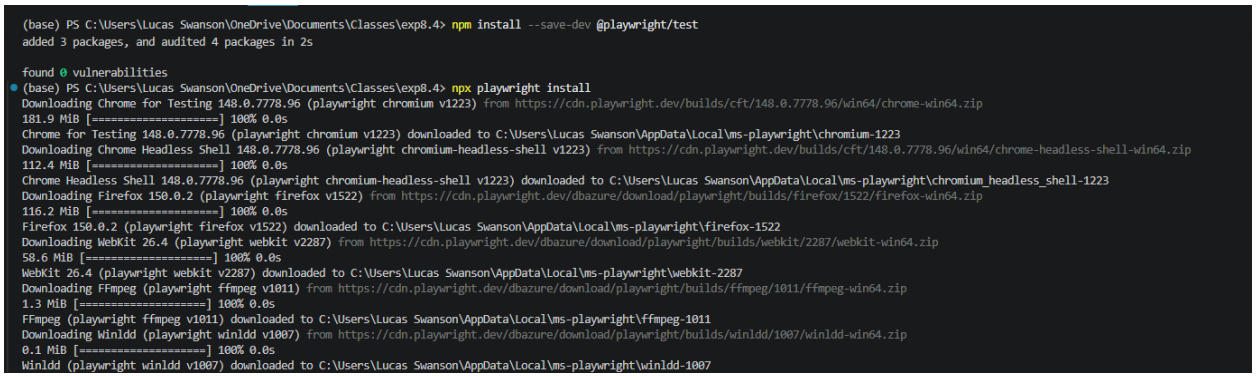
Use `npm install <pkg>` afterwards to install a package and save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (exp8.4)
version: (1.0.0)
description: A testing environment for the SpeedScore software
entry point: (index.js) index.html
test command: npm test
git repository: (https://github.com/CS561-OSU/exp8.4.git)
keywords: playwright
author: Lucas Swanson
license: (ISC)
type: (commonjs)
About to write to C:\Users\Lucas Swanson\OneDrive\Documents\Classes\exp8.4\package.json:

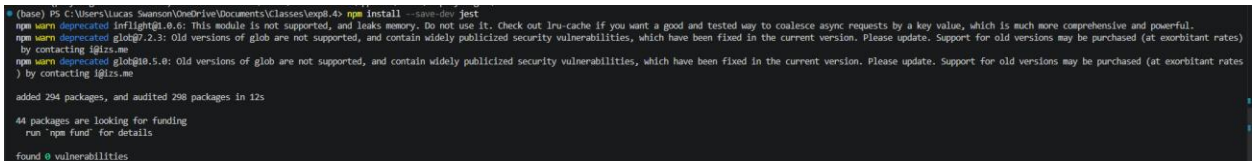
{
  "name": "exp8.4",
  "version": "1.0.0",
  "description": "A testing environment for the SpeedScore software",
  "main": "index.html",
  "scripts": {
    "test": "npm test"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/CS561-OSU/exp8.4.git"
  },
  "keywords": [
    "playwright"
  ],
  "author": "Lucas Swanson",
  "license": "ISC",
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/CS561-OSU/exp8.4/issues"
  },
  "homepage": "https://github.com/CS561-OSU/exp8.4#readme"
}


Is this OK? (yes) yes
```

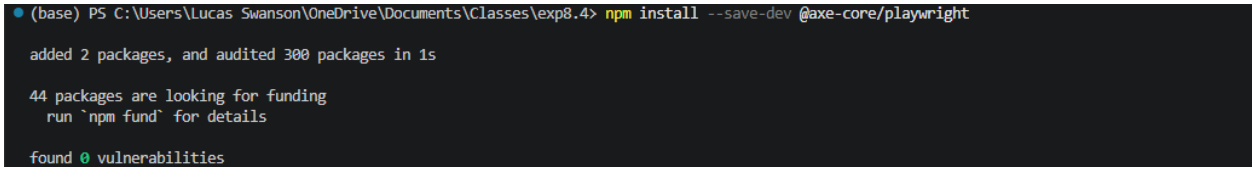
It will ask you some basic questions about descriptions and various items related to the project you are working on. Fill these in accordingly or follow along. The next thing you need to do is to install playwright with both npm and npx, the tool we will use for authoring test cases:



Next you need to install jest, a testing framework and test running tool:



The final item you need is axe-core/playwright, which is a node package that integrates the Axe accessibility engine with playwrite. It will allow you to run automated checks.



2. Test Execution

Here I have added three new sets of test cases for the following features:

- Edit profile functionality
- Side menu navigation functionality
- Dialog cancel / close functionality

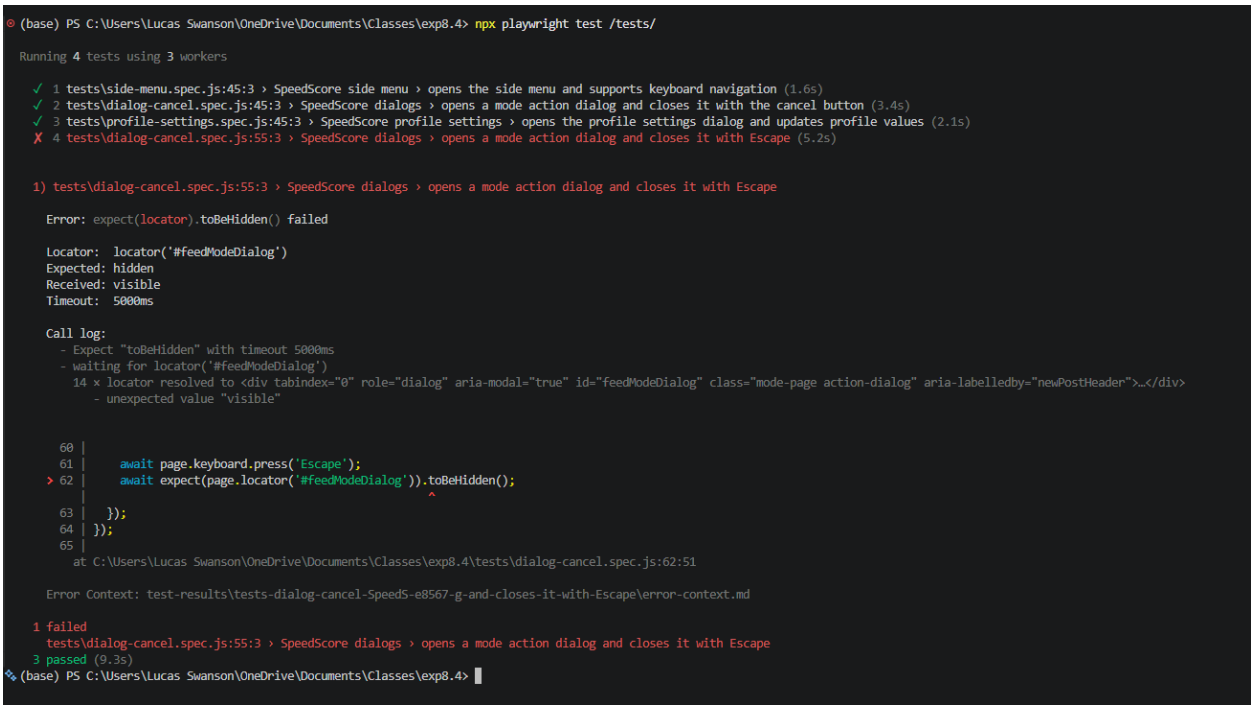
You run the test cases by specifying the npx package and the path to the test case you want to run:



You do this for any test suite you would like to test, or alter the path to encompass multiple test suites in order to test more than one item at a time.

3. Findings and analysis

Unfortunately one of the tests failed on the end to end tests. The side menu navigation, profile settings, and user information updates work just fine, but the ability to cancel or close pop-up menus with the escape key does not function.



4. Reflection

I think this tool does very well for its intended purpose, executing end-to-end tests to ensure functionality. Where other tests can execute on a unit or function level, this can help execute the whole suite together. This provides a much more robust solution than simple unit testing. However, it is limited by being only a testing framework, and is not able to point out anything much beyond a pass or a failed test. However, the difference between

Supertest and Playwright is in where the test is being executed. While Playwright uses the browser to execute tests, enabling you to perform test cases on visual items on the browser itself, Supertest is on the server side, or just using the website's API. This allows you to test without the use of a browser. This does not inherently make the testing frameworks better or worse than the others, simply different purposes from each other.